



BÁO CÁO BÀI THỰC HÀNH SỐ 2

Lập trình ngôn ngữ Assembly cơ bản

Môn học: NT209 - Lập trình Hệ thống

Sinh viên thực hiện	Lê Ngọc Phương Thảo - 23521467 Bùi Nguyễn Minh Anh - 24520086
Thời gian thực hiện	26/03/2026 – 01/04/2026
Số câu đã hoàn thành	4/4

NỘI DUNG BÁO CÁO

C.2.1 Chương trình in ra độ dài của một chuỗi cho trước (tối đa 9 ký tự)

Input: Chuỗi **msg** được khai báo sẵn trong section **.data**, tối đa 9 ký tự.

Output: Xuất ra màn hình độ dài của chuỗi (số ký tự - không tính ký tự null).

Giới hạn: Chuỗi có độ dài tối đa **9** ký tự

Mã nguồn (Source code):

```
.section .data
msg: .string "NT209UIT"
len: .int 0
enter: .byte 10 # trong ascii, byte có giá trị 10 nghĩa là xuống dòng

.section .text
.globl _start

_start:
    movl $len, %edi # $len là địa chỉ của len,
                    # lưu địa chỉ của len vào %edi
    subl $msg, %edi # trừ đi địa chỉ của msg
    subl $1, %edi   # trừ 1 đi để bỏ đi ký tự null

    addl $48, %edi  # cộng 48 để chuyển số thành ký tự
    movl %edi, len  # lưu độ dài của ký tự vào len

    movl $4, %eax   # sys_write
    movl $1, %ebx   # stdout
    movl $len, %ecx # đưa ký tự chứa độ dài lên vào thanh ghi
    movl $1, %edx   # in ra độ dài là 1 byte của chuỗi
    int $0x80       # gọi hệ thống thực thi

    movl $4, %eax   # sys_write
    movl $1, %ebx   # stdout
    movl $enter, %ecx # lưu địa chỉ ô nhớ enter vào thanh ghi
    movl $1, %edx   # in ra độ dài của chuỗi sẽ nhận
    int $0x80       # lời gọi hệ thống

    movl $1, %eax   # sys_exit
    movl $0, %ebx   #
    int $0x80       # gọi hệ thống thực thi
```

Kết quả chạy thử:

VD1: Với chuỗi msg = "NT209UIT", kết quả đầu ra như hình dưới

```
mab@mab-VirtualBox:~$ nano c21.s
mab@mab-VirtualBox:~$ as -o c21.o c21.s
mab@mab-VirtualBox:~$ ld -o c21 c21.o
mab@mab-VirtualBox:~$ ./c21
8
mab@mab-VirtualBox:~$
```

VD2: Với chuỗi msg = "Premium12", kết quả đầu ra như hình dưới

```
mab@mab-VirtualBox:~$ as -o c23.o c23.s
mab@mab-VirtualBox:~$ ld -o c23 c23.o
mab@mab-VirtualBox:~$ ./c23
9
mab@mab-VirtualBox:~$
```

Giải thích ý tưởng:

Bước 1 - Khai báo dữ liệu:

- Chuỗi **msg**: Chứa dữ liệu cần đếm.
- Biến **len**: Được đặt ngay sau chuỗi **msg**. Trong bộ nhớ, địa chỉ của **len** chính là vị trí kết thúc của chuỗi **msg** (tính cả ký tự null **\0**).
- Ký tự **enter** (10): xuống dòng sau khi in kết quả cho đẹp.

Bước 2 - Tính độ dài:

- Lấy địa chỉ của **len** (vị trí ngay sau chuỗi) lưu vào **%edi**.
- Trừ đi địa chỉ bắt đầu của **msg**. Kết quả là tổng số byte mà chuỗi đã chiếm
- Trừ tiếp 1 đơn vị để loại bỏ ký tự kết thúc null (**\0**).

Ví dụ: Nếu chuỗi là "UIT" (3 ký tự), trong bộ nhớ sẽ là **U, I, T, \0**. Hiệu địa chỉ sẽ là 4, trừ 1 còn 3.

Bước 3 - Chuyển đổi từ số sang ký tự (ASCII):

Máy tính lưu độ dài dưới dạng số nguyên (ví dụ số 8). Để in ra màn hình được số '8', buộc phải chuyển nó về mã ASCII.

- Cộng thêm 48 (mã ASCII của ký tự '0') vào con số vừa tính được.
- Kết quả được lưu vào ô nhớ `len` để sẵn sàng cho lệnh in.

Bước 4 - Gọi hệ thống để xuất kết quả

Để đưa số ra màn hình, ta thực hiện lời gọi hệ thống `sys_write`:

- `%eax = 4`: Mã lệnh in.
- `%ebx = 1`: In ra màn hình (STDOUT).
- `%ecx = $len`: Địa chỉ ô nhớ chứa ký tự số đã tính ở bước trên.
- `%edx = 1`: Chỉ in đúng 1 byte (vì độ dài tối đa là 9 ký tự).

C.2.2 Chương trình in ra chuỗi con của một chuỗi 10 ký tự

Input: Một chuỗi gồm 10 ký tự bất kỳ (số hoặc chữ không phân biệt hoa - thường), chỉ số (index) start và end để cắt chuỗi con.

Output: Xuất ra màn hình chuỗi con với các ký tự có index từ start đến end.

Mã nguồn (Source code):

```
.section .data
input_str: .string "Enter a string (10-chars): "
input_len = . -input_str

start_str: .string "Start index: "
start_len = . -start_str

end_str: .string "End index: "
end_len = . -end_str

newline: .byte 10

.section .bss
.lcomm input, 10
.lcomm start_idx, 1
.lcomm end_idx, 1

.section .text
.globl _start
_start:
    # Xuất chuỗi yêu cầu nhập input
```

```

movl $4, %eax          # sys_write
movl $1, %ebx          # stdout
movl $input_str, %ecx  # chuỗi in
movl $input_len, %edx  # độ dài chuỗi
int $0x80

# Nhập chuỗi input
movl $3, %eax          # sys_read
movl $0, %ebx          # stdin
movl $input, %ecx      # lưu giá trị nhập vào
movl $11, %edx         # độ dài tối đa của chuỗi input
                        # là (10 ký tự + \0)
int $0x80

# Xuất chuỗi yêu cầu nhập start index
movl $4, %eax
movl $1, %ebx
movl $start_str, %ecx
movl $start_len, %edx
int $0x80

# Nhập start index
movl $3, %eax
movl $0, %ebx
movl $start_idx, %ecx
movl $2, %edx
int $0x80

# Xuất chuỗi yêu cầu nhập end index
movl $4, %eax
movl $1, %ebx
movl $end_str, %ecx
movl $end_len, %edx
int $0x80

# Nhập end index
movl $3, %eax
movl $0, %ebx
movl $end_idx, %ecx
movl $2, %edx
int $0x80

# Chuyển start_idx thành số nguyên
xorl %ebx, %ebx        # xor hai giá trị giống nhau
                        # --> chuyển hết các bit về 0
                        # --> clear thanh ghi ebx
movb start_idx, %bl     # start_index = 1 byte = %bl

```

```

subb $0x30, %bl          # -> dễ dàng tính toán
                          # ký tự lưu ở %bl - '0' = int

# Chuyển end_idx thành số nguyên
xorl %eax, %eax          # clear thanh ghi eax
movb end_idx, %al
subb $0x30, %al

# Tính độ dài substring
subb %bl, %al             # al = end_idx - start_idx
addb $1, %al              # cộng thêm 1 để lấy cả ký tự cuối

xorl %edx, %edx          # xóa sạch thanh ghi edx
movb %al, %dl             # thanh ghi dl (edx)
                          # dùng để lưu độ dài chuỗi +
                          # trả lại thanh ghi eax
                          # để thực hiện lệnh khác

# Xác định vị trí bắt đầu của substring
movl $input, %ecx         # địa chỉ gốc của chuỗi
addl %ebx, %ecx           # ebx chứa start_idx
                          # ecx = địa chỉ gốc input
                          #      + start_idx
                          # ecx = địa chỉ bắt đầu in mới

# In ra chuỗi con
movl $4, %eax
movl $1, %ebx
int $0x80

# Xuống dòng
movl $4, %eax
movl $1, %ebx
movl $newline, %ecx
movl $1, %edx
int $0x80

# Exit code
movl $1, %eax
int $0x80

```

Kết quả chạy thử:

```
lnghpthao@DESKTOP-463NI94:~/nt209/lab2/C22$ as -o c22.o c22.s
lnghpthao@DESKTOP-463NI94:~/nt209/lab2/C22$ ld -o c22 c22.o
lnghpthao@DESKTOP-463NI94:~/nt209/lab2/C22$ ./c22
Enter a string (10-chars): 1234567890
Start index: 2
End index: 6
34567
lnghpthao@DESKTOP-463NI94:~/nt209/lab2/C22$ ./c22
Enter a string (10-chars): Hello ANTT
Start index: 0
End index: 4
Hello
lnghpthao@DESKTOP-463NI94:~/nt209/lab2/C22$ ./c22
Enter a string (10-chars): Hello 2026
Start index: 5
End index: 9
2026
```

Giải thích ý tưởng:

Bước 1 - Nhập input, start index và end index từ bàn phím.

Bước 2 - Chuyển các index nhập vào về dạng integer: Các index khi nhập từ bàn phím được lưu dưới dạng string. Để chuyển về dạng integer (số nguyên), nhóm sử dụng mã ASCII, lấy ký tự trừ đi giá trị 48 (tương ứng ký tự '0' trong bảng mã ASCII).

Bước 3 - Xác định địa chỉ bắt đầu mới của chuỗi cần in: Chương trình lấy địa chỉ gốc của chuỗi ban đầu cộng thêm start index. Kết quả là một địa chỉ trỏ thẳng vào ký tự bắt đầu của chuỗi con. Địa chỉ này được đưa vào thanh ghi **ecx**.

Bước 4 - Tính toán độ dài chuỗi cần in: Độ dài chuỗi cần in sẽ bằng end index - start index + 1. Giá trị này được lưu thẳng vào thanh ghi **edx** để hệ thống xác định được cần in bao nhiêu ký tự trong chuỗi.

Bước 5 - Thực hiện gọi ngắt và in ra chuỗi con.

C.2.3 Chương trình tính trung bình cộng của 4 số 1 chữ số

Input: 4 số 1 chữ số nhập vào từ bàn phím

Output: Giá trị trung bình cộng (lấy phần nguyên) của 4 số đã nhập.

Yêu cầu: Các số a, b, c, d nguyên và $0 \leq a, b, c, d < 10$.

Mã nguồn (Source code):

```
.section .data
msg: .string "Enter a number: "
len_msg = .- msg
res_msg: .string "tbc: "
len_res = .- res_msg
enter: .byte 10

.section .bss
.lcomm num, 2
.lcomm tbc, 1

.section .text
.globl _start

_start:
    movl $0, %esi                # lưu tổng giá trị vào
                                # thanh ghi %esi (khởi tạo bằng 0)

    # Nhập số thứ 1
    movl $4, %eax                # sys_write
    movl $1, %ebx                # stdout
    movl $msg, %ecx              # lưu địa chỉ của msg vào %ecx
    movl $len_msg, %edx          # số lượng byte cần in ra
    int $0x80                    # gọi hệ thống thực thi

    movl $3, %eax                # sys_read
    movl $0, %ebx                # stdin
    movl $num, %ecx              # lưu địa chỉ num vào %ecx
    movl $2, %edx                # đọc tối đa 2 byte(ký tự + enter)
    int $0x80

    movl $0, %eax                # xóa %eax để chuẩn bị tính toán
    movb (num), %al              # 1 byte ký tự vừa nhập vào %al
    subl $48, %eax               # chuyển từ ascii sang int
    addl %eax, %esi              # cộng dồn số thứ 1 vào %esi

    # Nhập số thứ 2
    movl $4, %eax                # sys_write
    movl $1, %ebx                # stdout
    movl $msg, %ecx              # lưu địa chỉ của msg vào %ecx
    movl $len_msg, %edx          # số lượng byte cần in ra
    int $0x80                    # gọi hệ thống thực thi

    movl $3, %eax                # sys_read
    movl $0, %ebx                # stdin
```



```

movl $num, %ecx      # lưu địa chỉ num vào %ecx
movl $2, %edx        # đọc tối đa 2 byte(ký tự + enter)
int $0x80

movl $0, %eax        # xóa %eax để chuẩn bị tính toán
movb (num), %al      # 1 byte ký tự vừa nhập vào %al
subl $48, %eax       # chuyển từ ascii sang int
addl %eax, %esi      # cộng dồn số thứ 2 vào %esi

# Nhập số thứ 3
movl $4, %eax        # sys_write
movl $1, %ebx        # stdout
movl $msg, %ecx      # lưu địa chỉ của msg vào %ecx
movl $len_msg, %edx  # số lượng byte cần in ra
int $0x80            # gọi hệ thống thực thi

movl $3, %eax        # sys_read
movl $0, %ebx        # stdin
movl $num, %ecx      # lưu địa chỉ num vào %ecx
movl $2, %edx        # đọc tối đa 2 byte(ký tự + enter)
int $0x80

movl $0, %eax        # xóa %eax để chuẩn bị tính toán
movb (num), %al      # 1 byte ký tự vừa nhập vào %al
subl $48, %eax       # chuyển từ ascii sang số nguyên
addl %eax, %esi      # cộng dồn số thứ 3 vào %esi

# Nhập số thứ 4
movl $4, %eax        # sys_write
movl $1, %ebx        # stdout
movl $msg, %ecx      # lưu địa chỉ của msg vào %ecx
movl $len_msg, %edx  # số lượng byte cần in ra
int $0x80            # gọi hệ thống thực thi

movl $3, %eax        # sys_read
movl $0, %ebx        # stdin
movl $num, %ecx      # lưu địa chỉ num vào %ecx
movl $2, %edx        # đọc tối đa 2 byte(ký tự + enter)
int $0x80

movl $0, %eax        # xóa %eax để chuẩn bị tính toán
movb (num), %al      # 1 byte ký tự vừa nhập vào %al
subl $48, %eax       # chuyển từ ascii sang số nguyên
addl %eax, %esi      # cộng dồn số thứ 4 vào %esi

```

```

# Tính trung bình cộng
movl %esi, %eax      # lưu tổng vào %eax để chia
movl $0, %edx        # xóa sạch %edx trước khi chia
movl $4, %ebx        # số chia là 4
divl %ebx            # chia %eax cho 4,
                    # kết quả phần nguyên nằm ở %eax

addl $48, %eax       # chuyển số sang ký tự ascii
movb %al, tbc        # lưu ký tự vào ô nhớ
                    # Trung bình cộng để in ra kết quả

movl $4, %eax
movl $1, %ebx
movl $res_msg, %ecx
movl $len_res, %edx
int $0x80

movl $4, %eax
movl $1, %ebx
movl $tbc, %ecx
movl $1, %edx
int $0x80

movl $4, %eax
movl $1, %ebx
movl $enter, %ecx
movl $1, %edx
int $0x80

# Thoát chương trình
movl $1, %eax        # sys_exit
movl $0, %ebx
int $0x80

```

Kết quả chạy thử:

```

mab@mab-VirtualBox:~$ as -o c233.o c233.s
mab@mab-VirtualBox:~$ ld -o c233 c233.o
mab@mab-VirtualBox:~$ ./c233
Enter a number: 3
Enter a number: 6
Enter a number: 9
Enter a number: 1
tbc: 4

```

Giải thích ý tưởng:

Bước 1 - Khai báo và chuẩn bị vùng nhớ:

- Vùng dữ liệu (.data): Chứa các chuỗi thông báo cố định như "Enter a number: " và "tbc: ".

- Vùng bộ nhớ đệm (.bss): Cấp phát ô nhớ **num** (2 byte) để nhận dữ liệu nhập từ bàn phím và **average** (1 byte) để chứa kết quả sau cùng.

Bước 2 - Nhập liệu và tích lũy tổng:

- In lời nhắc: Dùng **sys_write** (thanh ghi **%eax = 4**) để hiện chữ lên màn hình.
- Đọc dữ liệu: Dùng **sys_read** (thanh ghi **%eax = 3**) để máy dừng lại chờ bạn gõ số.
- Chuyển đổi kiểu dữ liệu: Dữ liệu từ bàn phím luôn là ký tự ASCII. Ta phải lấy byte đó ra (**movb**), trừ đi 48 (**subl \$48**) để biến ký tự '5' thành con số 5 thật sự.
- Cộng dồn: Dùng thanh ghi **%esi** làm "kho chứa", cứ mỗi lần có số mới thì cộng thêm vào (**addl**).

Bước 3 - Thực hiện phép chia lấy phần nguyên:

Sau khi đã có tổng của 4 số nằm trong **%esi**, thực hiện phép chia:

- Chuyển tổng sang **%eax** và xóa sạch **%edx** (bắt buộc để phép chia 32-bit không bị lỗi).
- Dùng lệnh **divl** chia cho 4. Theo kiến trúc x86, kết quả phần nguyên sẽ tự động được lưu vào thanh ghi **%eax**.

Bước 4 - Chuyển đổi ngược và xuất kết quả:

- Mã hóa: Lấy kết quả phần nguyên trong **%eax** cộng lại với 48 (**addl \$48**) để biến số thành ký tự có thể hiển thị được.
- In kết quả: Gọi **sys_write** để in chuỗi "tbc: " và ký tự số vừa tính được.
- Xuống dòng: Cuối cùng là in ký tự **enter** (mã 10) để con trỏ chuột nhảy xuống dòng mới.

Bước 5 - Kết thúc chương trình:

Dùng **sys_exit** (thanh ghi **%eax = 1**) kết thúc chương trình.

C.2.4 Mã hóa Caesar cho chuỗi gồm 4 ký tự in thường

Input: Chuỗi gồm 4 ký tự chữ in thường, số bước dịch **n** (0 – 9), trong đó với 1 ký tự **x** thì $'a' \leq x + n \leq 'z'$.

Output: Xuất ra chuỗi ký tự đã được mã hóa Caesar với bước dịch **n**.

Nâng cao: Xử lý được trường hợp xoay vòng khi kết quả sau khi dịch vượt quá ký tự 'z'. Ví dụ $'z' + 1 = 'a'$, $'z' + 2 = 'b'$.

Mã nguồn (Source code):

```

.section .data
input_str: .string "Nhap chuoi (4 ky tu): "
input_len = . -input_str

n_str: .string "Nhap n (0-9): "
n_len = . -n_str

newline: .byte 10


.section .bss
.lcomm input, 4
.lcomm n, 1


.section .text
.globl _start
_start:
    # Xuất chuỗi yêu cầu nhập input
    movl $4, %eax           # sys_write
    movl $1, %ebx           # stdout
    movl $input_str, %ecx   # chuỗi in
    movl $input_len, %edx   # độ dài chuỗi
    int $0x80

    # Nhập chuỗi input
    movl $3, %eax           # sys_read
    movl $0, %ebx           # stdin
    movl $input, %ecx       # lưu giá trị vào biến input
    movl $5, %edx           # độ dài tối đa (4 ký tự + \0)
    int $0x80

    # Xuất chuỗi yêu cầu nhập n
    movl $4, %eax
    movl $1, %ebx
    movl $n_str, %ecx
    movl $n_len, %edx
    int $0x80

    # Nhập n
    movl $3, %eax
    movl $0, %ebx
    movl $n, %ecx
    movl $2, %edx
    int $0x80

    # Chuyển n thành số nguyên (integer)

```

```

xorl %eax, %eax          # clear thanh ghi eax
movb n, %al
subb $0x30, %al          # ký tự trong %al - '0' = int

# Mã hóa ký tự thứ 1 trong chuỗi
movl $input, %ebx        # ebx trỏ đến địa chỉ chuỗi
movb (%ebx), %cl         # lấy ký tự thứ 1 trong input
addb %al, %cl            # dịch n bước so với ký tự x

# Kiểm tra ký tự có cần xoay vòng hay không
movb %cl, %dl
subb $123, %dl           # kiểm tra > 'z' hay không
sarb $7, %dl             # dịch phải 7 bit để lấy bit dấu
                        # nếu >= 123: %dl = 0x00,
                        # nếu < 123: %dl = 0xFF
notb %dl                 # đảo dấu để AND với 26
                        # nếu >= 123: %dl = 0xFF
                        # nếu < 123: %dl = 0x00
andb $26, %dl            # 0xFF and 26 = 26,
                        # 0x00 and 26 = 0
subb %dl, %cl            # nếu >= 123: -26 (xoay vòng)
                        # nếu < 123: - 0 (giữ nguyên)
movb %cl, (%ebx)

# Mã hóa ký tự thứ 2 trong chuỗi
incl %ebx
movb (%ebx), %cl         # lấy ký tự thứ 2 trong input
addb %al, %cl

# Kiểm tra ký tự có cần xoay vòng hay không
movb %cl, %dl
subb $123, %dl
sarb $7, %dl
notb %dl
andb $26, %dl
subb %dl, %cl
movb %cl, (%ebx)

# Mã hóa ký tự thứ 3 trong chuỗi
incl %ebx
movb (%ebx), %cl         # lấy ký tự thứ 3 trong input
addb %al, %cl

# Kiểm tra ký tự có cần xoay vòng hay không
movb %cl, %dl
subb $123, %dl
sarb $7, %dl

```

```

notb %dl
andb $26, %dl
subb %dl, %cl
movb %cl, (%ebx)

# Mã hóa ký tự thứ 4 trong chuỗi
incl %ebx
movb (%ebx), %cl      # lấy ký tự thứ 4 trong input
addb %al, %cl

# Kiểm tra ký tự có cần xoay vòng hay không
movb %cl, %dl
subb $123, %dl
sarb $7, %dl
notb %dl
andb $26, %dl
subb %dl, %cl
movb %cl, (%ebx)

# Xuất chuỗi đã được mã hóa Caesar
movl $4, %eax
movl $1, %ebx
movl $input, %ecx
movl $4, %edx
int $0x80

# Xuống dòng
movl $4, %eax
movl $1, %ebx
movl $newline, %ecx
movl $1, %edx
int $0x80

# Kết thúc chương trình
movl $1, %eax
int $0x80

```

Kết quả chạy thử (phần cơ bản):

```
lmgphthao@DESKTOP-463NI94:~/nt209/lab2/C24$ as -o c24.o c24.s
lmgphthao@DESKTOP-463NI94:~/nt209/lab2/C24$ ld -o c24 c24.o
lmgphthao@DESKTOP-463NI94:~/nt209/lab2/C24$ ./c24
Nhap chuoi (4 ky tu): abcd
Nhap n (0-9): 1
bcde
lmgphthao@DESKTOP-463NI94:~/nt209/lab2/C24$ ./c24
Nhap chuoi (4 ky tu): abcd
Nhap n (0-9): 5
fghi
lmgphthao@DESKTOP-463NI94:~/nt209/lab2/C24$ ./c24
Nhap chuoi (4 ky tu): mngh
Nhap n (0-9): 8
uvop
```

Kết quả chạy thử (phần nâng cao):

```
lmgphthao@DESKTOP-463NI94:~/nt209/lab2/C24$ as -o c24.o c24.s
lmgphthao@DESKTOP-463NI94:~/nt209/lab2/C24$ ld -o c24 c24.o
lmgphthao@DESKTOP-463NI94:~/nt209/lab2/C24$ ./c24
Nhap chuoi (4 ky tu): abyz
Nhap n (0-9): 1
bcza
lmgphthao@DESKTOP-463NI94:~/nt209/lab2/C24$ ./c24
Nhap chuoi (4 ky tu): name
Nhap n (0-9): 9
wjvn
lmgphthao@DESKTOP-463NI94:~/nt209/lab2/C24$ ./c24
Nhap chuoi (4 ky tu): xywz
Nhap n (0-9): 7
efdg
```

Giải thích ý tưởng:

Bước 1 - Nhập input (chuỗi 4 ký tự) và số bước dịch n từ bàn phím

Bước 2 - Chuyển đổi n về dạng số nguyên (integer): Giá trị n khi nhập vào được lưu dưới dạng string. Lấy giá trị này trừ đi 48 (mã ASCII của ký tự '0') để thu được giá trị số nguyên để tính toán dịch chuyển.

Bước 3 - Thực hiện dịch chuyển ký tự: Chương trình lần lượt lấy thủ công ký tự trong chuỗi và cộng trực tiếp với giá trị n đã chuyển đổi. Kết quả tạm thời này được lưu vào thanh ghi **cl** để chuẩn bị cho bước kiểm tra xoay vòng.

Bước 4 - Xử lý xoay vòng: Để xử lý xoay vòng, trước hết ta kiểm tra xem liệu ký tự có vượt qua khỏi ký tự 'z' hay không.

- Lấy ký tự sau khi dịch trừ cho 123 (giá trị ngay sau 'z').
- Sử dụng lệnh dịch phải số học 7-bit để trích xuất bit dấu. Nếu ký tự vượt quá 'z', kết quả của phép dịch này sẽ có giá trị là 0.

- Dùng lệnh **not** và **and** với số 26 (số lượng kí tự trong bảng chữ cái) để tính toán giá trị cần trừ. Nếu vượt ngưỡng 'z', giá trị hiệu chỉnh sẽ là 26, ngược lại sẽ là 0.
- Lấy ký tự đã dịch trừ đi giá trị hiệu chỉnh này để đưa ký tự về đúng khoảng 'a' - 'z'.

Bước 5 - Lặp lại từ bước 3 cho từng ký tự còn lại.

Bước 6 - Lưu kết quả và in chuỗi đã mã hóa.